# Приближённый поиск ближайших соседей (ANN)

## Table of contents

Приближённый поиск ближайших соседей (ANN) . . . . . 1
HNSW (Hierarchical Navigable Small World) . . . . . 1
LSH (Locality-Sensitive Hashing) . . . . . 6
Векторные БД в продакшене . . . . . 10

## Приближённый поиск ближайших соседей (ANN)

Точный поиск k ближайших соседей (k-NN) требует сравнения запроса со всеми векторами в базе. Для больших коллекций (миллионы-миллиарды векторов) это неприемлемо медленно. Приближённый поиск (ANN) строит специальные индексы, которые находят "почти ближайших" соседей за O(log n) вместо O(n).

Компромисс ANN: - **Recall** (полнота): доля истинно ближайших соседей среди найденных (обычно 90-99%) - **Latency** (задержка): время поиска (миллисекунды вместо секунд) - **Memory** (память): индекс требует дополнительной памяти сверх векторов

Основные подходы к ANN: - **Графовые индексы** (HNSW, NSW) — навигация по графу близости - **Хеш-индексы** (LSH) — похожие векторы попадают в одни корзины - **Квантизация** (IVF, PQ) — разбиение пространства на кластеры

### HNSW (Hierarchical Navigable Small World)

**Основная идея:**

Проблема: найти k ближайших векторов к запросу. Brute-force перебирает все n векторов в базе, вычисляя расстояние до каждого (O(n × d), где d — размерность). Для миллионов векторов это слишком медленно.

Решение: строим многослойный граф: - Верхние слои — «магистральные» длинные рёбра для быстрого перехода к нужной области - Нижние слои — локальные короткие рёбра для точного поиска

Алгоритм поиска: спускаемся сверху вниз, на каждом слое жадно переходим к ближайшему соседу. Сложность **sublinear** — эмпирически близко к O(log n) при правильном выборе параметров M и ef (вместо O(n) у brute-force).

**Поддерживаемые метрики:** - **Евклидово расстояние (L2)** — обычное расстояние между точками в пространстве - **Косинусное сходство** — мера угла между векторами (чем меньше угол, тем ближе) - ⚠️ Важно: векторы должны быть нормализованы (длина = 1)

**Ограничение:** HNSW показывает лучшие результаты на метрических расстояниях (L2 или косинус через нормализацию). Скалярное произведение (inner product) без нормализации не является метрикой (не выполняется неравенство треугольника), что может снизить recall.

Варианты работы с inner product: - Нормализовать векторы → сводится к косинусному расстоянию - Использовать специальные трансформации (например, MIPS→L2) - Применить другие индексы (например, IVF с inner product)

Go-псевдоинтерфейс и оценки:

```go
// HNSW: графовый индекс ближайших соседей.
type HNSW interface {
    // Add: вставка с поиском входной точки.
    // O(M * log n) амортизировано.
    Add(id int, vec []float32)

    // Search: greedy поиск по слоям.
    // O(M * log n) амортизировано.
    Search(query []float32, k int) []int
}
```

**Структура данных:**

Многослойный граф: - Вершины = векторы - Рёбра = связи между близкими векторами (не более M соседей на вершину) - Слои L0, L1, L2, ...: - L0 (нижний) содержит все векторы - L1 содержит случайное подмножество (~50% от L0) - L2 содержит случайное подмножество L1 (~25% от L0) - И так далее до единственной вершины на верхнем слое (entry point)

Каждая вершина хранит:

```go
type Node struct {
    id     int
    vector []float32
    level  int           // максимальный слой, где есть эта вершина
    layers [][]int       // соседи на каждом слое: layers[l] = [id1, id2, ...]
}
```

Параметр M (макс. число рёбер): обычно 16-32 - Больше M → точнее поиск, но медленнее вставка и больше памяти - Меньше M → быстрее, но хуже recall

**Добавление вектора:**

```
1. Генерируем случайный уровень l для новой вершины v
   (экспоненциальное распределение: p(l) = (1/2)^l)
```

```
2. Начинаем с entry_point на верхнем слое
   Для каждого слоя сверху вниз до l:
     - Жадный поиск: идём к ближайшему соседу, пока улучшается расстояние
     - Запоминаем найденную вершину как точку входа для следующего слоя

3. На слоях от l до 0 (где v присутствует):
     - Ищем efConstruction ближайших соседей (efConstruction ~ 100-200)
     - Выбираем из них M лучших (эвристика: разнообразие + близость)
     - Добавляем рёбра v → соседи
     - Для каждого соседа: добавляем ребро сосед → v
       Если у соседа стало > M рёбер, обрезаем до M (оставляем лучшие)

4. Если l выше текущего entry_point, обновляем entry_point = v
```

**Поиск k ближайших:**

```
1. Начинаем с entry_point на верхнем слое
   curr = entry_point

2. Для каждого слоя сверху вниз до L0:
     - Жадный спуск: пока есть улучшение
       * Смотрим всех соседей curr на этом слое
       * Переходим к ближайшему к query
       * Повторяем, пока curr не перестанет улучшаться
     - Переходим на слой ниже с текущим curr

3. На L0 (нижний слой):
    - Расширяем поиск: поддерживаем priority queue из ef кандидатов (ef ~ 50-500)
     - Для каждого кандидата смотрим его соседей
     - Добавляем в queue, если улучшают расстояние
     - Останавливаемся, когда нет новых улучшений
     - Возвращаем топ-k из queue
```

**Пример 1: Структура многослойного графа**

9 точек в 2D-пространстве с координатами:

```
A(0,0)  B(1,0)  C(2,0)  D(3,0)  E(4,0)
        F(2,1)  G(3,1)  H(4,1)  I(5,1)
```

Граф построен с параметром M=3 (макс. 3 соседа на вершину на слое):

**Слой L2 (entry point):**

```
A: [D]           // только одно дальнее ребро-магистраль
D: [A]
```

**Слой L1:**

```
A: [C, D]
C: [A, D, F]
D: [A, C, G]
G: [D]
```

**Слой L0 (все векторы):**

```
A: [B, C]
B: [A, C]
C: [A, B, D, F]
D: [C, E, G]
E: [D, H]
F: [C, G]
G: [D, F, H]
H: [E, G, I]
I: [H]
```

Ключевое свойство: **одна вершина может быть на нескольких слоях, но списки соседей разные**. Например, D есть на L2, L1 и L0 с разными ребрами.

**Пример 2: Поиск ближайшего к query(5.2, 1.1)**

Ожидаемый результат: I(5,1) — ближайшая точка.

**Фаза 1: Жадный спуск по слоям**

```
L2: Старт с entry=A(0,0)
    - dist(query, A) = 5.3
    - Соседи A на L2: [D]
    - dist(query, D) = 2.2  →  D ближе, переходим
    - Соседи D на L2: [A]  →  нет улучшений
    → Спускаемся на L1 с curr=D

L1: curr=D(3,0)
    - Соседи D на L1: [A, C, G]
    - dist(query, A) = 5.3
    - dist(query, C) = 3.3
    - dist(query, G) = 2.2  →  G ближе, переходим
    - Соседи G на L1: [D]  →  нет улучшений
```

```
    → Спускаемся на L0 с curr=G

L0: curr=G(3,1)
    - Соседи G на L0: [D, F, H]
    - dist(query, D) = 2.3
    - dist(query, F) = 3.2
    - dist(query, H) = 1.2  →  H ближе, переходим
    - Соседи H на L0: [E, G, I]
    - dist(query, I) = 0.2  →  I ближе, переходим
    - Соседи I на L0: [H]  →  нет улучшений
    → Жадный поиск остановился на I
```

**Фаза 2: Расширенный поиск с ef (на L0)**

```
Поддерживаем очередь кандидатов размера ef=10
Стартуем с I, добавляем соседей I → [H]
Из H берем соседей → [E, G, I]
Из E → [D, H], из G → [D, F, H]
Собрали кандидатов: {I, H, E, G, D, F}
Вычисляем точные расстояния:
  dist(query, I) = 0.20
  dist(query, H) = 1.20
  dist(query, E) = 1.56
  ...
Топ-3 результата: [I, H, E]
```

**Вывод:** Жадный спуск быстро приводит в нужную область (за O(log n) переходов), а расширенный поиск с ef добирает соседей для точности.

**Пример 3: Вставка новой точки v(3.5, 0.8)**

```
1. Генерируем уровень v: level(v) = 1 (случайно)
   → v будет присутствовать на L1 и L0

2. Поиск точки входа (как в примере 2):
   L2: A → D  (curr=D)
   L1: D → G  (curr=G)
   L0: G (уже близко к v)

3. Вставка на L1:
   - Ищем efConstruction=5 ближайших к v на L1
   - Кандидаты: {D(dist=0.95), G(dist=0.54), C(dist=1.73), A(dist=3.62)}
   - Выбираем M=3 лучших: [G, D, C]
```

```
   - Добавляем рёбра: v → [G, D, C]
   - Обратные рёбра: G.neighbors_L1 += [v], D.neighbors_L1 += [v], C.neighbors_L1
+= [v]
   - Проверяем: у D теперь [A, C, G, v] = 4 соседа > M
     Обрезаем до M=3 лучших: D.neighbors_L1 = [C, G, v] (убрали дальнего A)

4. Вставка на L0:
   - Аналогично: v → [G, D, F]
   - Обновляем обратные рёбра

Итоговый граф:

L2: без изменений (v не попал на L2)

L1:
A: [C]              // изменено: убрали D
C: [A, D, F, v]     // добавили v
D: [C, G, v]        // изменено: убрали A, добавили v
G: [D, v]           // добавили v
v: [G, D, C]        // новая вершина

L0:
D: [C, E, G, v]     // добавили v
F: [C, G, v]        // добавили v
G: [D, F, H, v]     // добавили v (обрезали до M=3, если нужно)
v: [G, D, F]        // новая вершина
(остальные без изменений)
```

**Ключевой момент:** вставка может изменить граф (обрезка рёбер при превышении M), поэтому качество графа зависит от порядка вставок. Практика: случайный порядок или периодическая переиндексация.

**Практическое применение:** - Семантический поиск с большими коллекциями (миллионы векторов) - Поиск ближайших соседей в рекомендациях - Быстрый приближённый поиск (ANN, Approximate Nearest Neighbors) с управляемой точностью (recall ~95% за 1-5 мс)

## LSH (Locality-Sensitive Hashing)

**Основная идея:**

Проблема: как быстро найти похожие векторы без построения сложных графов (как в HNSW)?

Решение: хешируем векторы специальным образом — **похожие векторы с высокой вероятностью попадают в один бакет хеш-таблицы**. Вместо сравнения со всеми векторами в базе, сравниваем только с теми, что попали в те же бакеты.

Ключевое свойство locality-sensitive hash-функции:

```
Если distance(v1, v2) мала → P(hash(v1) == hash(hash(v2)) велика
Если distance(v1, v2) велика → P(hash(v1) == hash(v2)) мала
```

Go-псевдоинтерфейс и оценки:

```go
// LSH: вероятностный поиск ближайших.
type LSH interface {
    // Add: добавить вектор в бакеты.
    // O(L * k * d), L — число таблиц, k — число хешей.
    Add(id int, vec []float32)

    // Search: искать в бакетах запроса.
    // O(L * k * d + c * d), c — кандидаты.
    Search(query []float32, k int) []int
}
```

**Структура данных:**

LSH использует **L независимых хеш-таблиц**, каждая со своей хеш-функцией:

```
Таблица 1: hash_1(v) → бакет в таблице 1
Таблица 2: hash_2(v) → бакет в таблице 2
...
Таблица L: hash_L(v) → бакет в таблице L
```

Параметры: - **L** (число таблиц) — обычно 10-50. Больше L ⟶ выше recall, но медленнее - **k** (число хешей на таблицу) — обычно 3-10. Больше k ⟶ меньше коллизий, но жёстче фильтр

**Random Projection LSH (для косинусного расстояния):**

Самая популярная схема для векторных поисков. Используем **случайные гиперплоскости**.

Одна хеш-функция:

```
1. Генерируем случайный вектор r (нормальное распределение)
2. hash_bit(v) = sign(v · r)  // скалярное произведение
   = 1, если v · r ≥ 0
   = 0, если v · r < 0
```

Гиперплоскость r делит пространство на 2 полупространства. Векторы с одной стороны ⟶ hash=1, с другой ⟶ hash=0.

Для одной таблицы используем **k случайных гиперплоскостей** $r_1, r_2, \ldots, r_k$:

```
hash(v) = [hash_bit(v, r₁), hash_bit(v, r₂), ..., hash_bit(v, rₖ)]
        = битовая строка длины k
        = номер бакета (от 0 до 2^k - 1)
```

**Пример: 6 векторов в 2D**

Векторы (нормализованные):

```
A(1.0, 0.2)   B(0.9, 0.3)   C(0.3, 0.9)
D(0.2, 1.0)   E(-0.7, 0.7)  F(-1.0, -0.1)
```

Строим LSH с L=2 таблицами, k=2 хешей на таблицу.

**Таблица 1:**

```
Случайные векторы:
  r₁ = (0.6, 0.8)   // гиперплоскость наклонена вправо-вверх
  r₂ = (-0.8, 0.6)  // гиперплоскость наклонена влево-вверх

Вычисляем хеши:
  A(1.0, 0.2): A·r₁ = 0.76 > 0 → bit₁=1, A·r₂ = -0.68 < 0 → bit₂=0
               hash(A) = [1,0] = бакет 2

  B(0.9, 0.3): B·r₁ = 0.78 > 0 → bit₁=1, B·r₂ = -0.54 < 0 → bit₂=0
               hash(B) = [1,0] = бакет 2

  C(0.3, 0.9): C·r₁ = 0.90 > 0 → bit₁=1, C·r₂ = 0.30 > 0 → bit₂=1
               hash(C) = [1,1] = бакет 3

  D(0.2, 1.0): D·r₁ = 0.92 > 0 → bit₁=1, D·r₂ = 0.44 > 0 → bit₂=1
               hash(D) = [1,1] = бакет 3

  E(-0.7, 0.7): E·r₁ = 0.14 > 0 → bit₁=1, E·r₂ = 0.98 > 0 → bit₂=1
                hash(E) = [1,1] = бакет 3

  F(-1.0, -0.1): F·r₁ = -0.68 < 0 → bit₁=0, F·r₂ = 0.74 > 0 → bit₂=1
                 hash(F) = [0,1] = бакет 1

Бакеты таблицы 1:
  Бакет 1: [F]
```

```
  Бакет 2: [A, B]       ← A и B близки → попали в один бакет ✓
  Бакет 3: [C, D, E]    ← C, D близки → попали в один бакет ✓
```

**Таблица 2** (другие случайные $r_3$, $r_4$):

```
Аналогично строим вторую таблицу с другими гиперплоскостями.
Из-за случайности могут быть другие группировки.
```

**Поиск query(0.85, 0.4):**

```
1. Вычисляем hash(query) для каждой таблицы:
   Таблица 1: hash(query) = [1,0] = бакет 2
   Таблица 2: hash(query) = [1,1] = бакет 3 (допустим)

2. Собираем кандидатов из соответствующих бакетов:
   Из таблицы 1, бакет 2: {A, B}
   Из таблицы 2, бакет 3: {C, D, E} (допустим)
   Объединение: {A, B, C, D, E}

3. Точно вычисляем расстояния только для кандидатов:
   dist(query, A) = 0.15
   dist(query, B) = 0.10  ← ближайший
   dist(query, C) = 0.58
   dist(query, D) = 0.72
   dist(query, E) = 1.52

4. Возвращаем топ-k: [B, A]

Не проверяли F → сэкономили вычисления!
```

**Почему несколько таблиц (L > 1)?**

С одной таблицей можем пропустить близкие векторы (низкий recall): - Если query и близкий вектор v случайно попали в разные бакеты ⟶ v потеряли

С L таблицами: - Достаточно попасть в один бакет **хотя бы в одной таблице** - Вероятность пропустить близкий вектор снижается экспоненциально

Компромисс: - Больше L ⟶ выше recall, но больше кандидатов ⟶ медленнее - Больше k ⟶ меньше ложных срабатываний, но можем пропустить близкие векторы

**Практическое применение:** - Дедупликация больших коллекций (поиск почти-дубликатов документов) - Первичная фильтрация в поисковых системах (миллиарды векторов) - Сценарии, где допустим trade-off recall/скорость (recall ~70-80%) -

Предфильтрация перед точным ранжированием (LSH → сузили до 1000 кандидатов → точный rerank)

**Сравнение LSH и HNSW:**

| Характеристика | LSH | HNSW |
|---|---|---|
| Recall (точность) | 70-85% — вероятностный | 90-99% — детерминированный |
| Скорость поиска | Субмиллисекунды — только хеширование | 1-5 мс — обход графа |
| Память на вектор | Компактно: несколько битов на таблицу | Граф: M×L связей (обычно 100-500 байт) |
| Скорость вставки | O(L × k) — очень быстро, только хеширование | O(M log n) — медленнее, поиск + обновление графа |
| Параметры настройки | L (таблицы), k (хеши на таблицу) | M (макс. связей), ef (размер очереди) |
| Когда использовать | • Огромные базы (миллиарды векторов)• Допустим низкий recall• Нужна максимальная скорость• Предфильтрация кандидатов | • Средние/большие базы (миллионы)• Нужен высокий recall (>95%)• Можно потратить 1-10 мс• Продакшен-поиск с гарантиями |

**Когда выбирать комбинацию:**

```
LSH (грубый фильтр) → HNSW (точный rerank)
  ↓                       ↓
1 млрд векторов → 10K кандидатов → топ-100 результатов
```

## Векторные БД в продакшене

**Специализированные векторные базы данных:**

**Weaviate** (open-source): - Индекс: HNSW по умолчанию - Фичи: GraphQL API, гибридный поиск (вектор + фильтры), multi-tenancy - Когда: нужен self-hosted с богатым API и фильтрацией - Пример use case: семантический поиск по документам с фильтрами по метаданным (дата, автор, теги)

**Pinecone** (managed SaaS): - Индекс: проприетарная оптимизация поверх HNSW - Фичи: автоматическое шардирование, репликация, serverless масштабирование - Когда: не хочется управлять инфраструктурой, нужна отказоустойчивость out-of-the-box - Пример use case: recommendation engine с миллионами пользователей

**Qdrant** (open-source, Rust): - Индекс: HNSW с оптимизациями - Фичи: payload-фильтры, sparse vectors, quantization для экономии памяти - Когда: нужен производительный self-hosted с низким overhead - Пример use case: поиск похожих изображений в e-commerce с фильтрацией по категориям

**Где встречается в реальных продуктах:**

**RAG (Retrieval-Augmented Generation):**

```
Вопрос → векторный поиск по базе знаний → контекст для LLM → ответ
```

- Чат-боты техподдержки (поиск по тикетам)
- Корпоративные ассистенты (поиск по внутренним документам)
- Научные поисковики (поиск релевантных статей)

**E-commerce и контент:** - Поиск похожих товаров по изображению (Zalando, ASOS) - Рекомендации контента (YouTube, Spotify embeddings) - Дедупликация (поиск дубликатов товаров у разных продавцов)

**Мониторинг и безопасность:** - Поиск аномалий в логах через embeddings (Elastic) - Детекция фрода по паттернам транзакций - Поиск похожих инцидентов в системах мониторинга